

芝士架构公共知识红宝书

2026 年 10 月终稿 - 芝士架构凯恩编辑整理 v6.0.0

系统架构设计师公共基础知识



目录

芝士架构公共知识红宝书	1
目录	2
前言	8
使用说明	9
常见问题	10
在线刷题	13
红宝书 6.0 章节安排思路	14
1. 软件工程概述（重点★★★★★）	15
1.1. 软件工程相关定义（次重点★★★★☆☆）	15
1.2. 软件开发方法（重点★★★★★）	15
1.3. 软件过程/生命周期模型（重点★★★★★）	24
1.4. 软件过程管理（次重点★★★★☆☆）	33
2. 需求工程（超级重点★★★★★）	35
2.1. 需求获取（次重点★★★★☆☆）	36
2.2. 需求分析简介（非重点☆☆☆☆☆）	37
2.3. 结构化分析方法/建模 SA（超级重点★★★★★）	37
2.4. 面向对象分析方法/建模（超级重点★★★★★）	43
2.5. 建模语言（重点★★★★★）	49
2.6. 需求定义（非重点☆☆☆☆☆）	56
2.7. 需求验证（非重点☆☆☆☆☆）	56
2.8. 需求管理（重点★★★★★）	57

3. 系统设计（超级重点★★★★★）	62
3.1. 系统设计在软件工程中的位置（非重点★☆☆☆☆）	62
3.2. 系统设计概述（次重点★★★★☆）	63
3.3. 处理流程设计概述（次重点★★★★☆）	64
3.4. 面向对象设计（重点★★★★★）	69
3.5. 结构化设计（重点★★★★★）	71
3.6. 设计模式（次重点★★★★☆）	74
4. 系统架构设计（超级重点★★★★★）	75
4.1. 基本定义（重点★★★★★）	76
4.2. 软考中的 4+1 视图（重点★★★★★）	76
4.3. 软件架构设计与生命周期（非重点☆☆☆☆☆）	77
4.4. 基于架构的软件开发方法 ABSD（次重点★★★★☆）	78
4.5. 软件架构风格（超级重点★★★★★）	80
4.6. 软件架构复用（重点★★★★★）	84
4.7. 特定领域软件体系结构 DSSA（次重点★★★★☆）	85
4.8. 系统质量属性与架构评估（重点★★★★★）	86
5. 软件测试（次重点★★★★☆）	96
5.1. 测试分类方法（重点★★★★★）	96
5.2. 功能测试（次重点★★★★☆）	100
5.3. 性能测试（次重点★★★★☆）	100
5.4. 安全测试（次重点★★★★☆）	101
5.5. 软件测试模型 W/H/X 模型（次重点★★★★☆）	102
6. 系统运行与维护（次重点★★★★☆）	102

6.1. 运维技术指标（次重点★★★☆☆）	102
6.2. 系统运行管理（非重点☆☆☆☆☆）	103
6.3. 系统故障管理（非重点☆☆☆☆☆）	104
6.4. 软件系统维护（次重点★★★☆☆）	105
6.5. 遗留系统处置（次重点★★★☆☆）	106
6.6. 新旧系统转换（次重点★★★☆☆）	107
7. 系统可靠性（次重点★★★☆☆）	108
7.1. 软件可靠性设计（次重点★★★☆☆）	108
7.2. 软件可靠性的定量描述（重点★★★★★）	109
7.3. 影响软件可靠性的 5 大因素（非重点☆☆☆☆☆）	111
7.4. 程序复杂程度的定量度量 McCabe 方法（次重点★★★☆☆）	112
7.5. 串并联系统（重点★★★★★）	114
8. 项目管理（次重点★★★☆☆）	115
8.1. 范围管理（次重点★★★☆☆）	115
8.2. 进度管理（重点★★★★★）	116
8.3. 成本管理（次重点★★★☆☆）	118
8.4. 配置管理（次重点★★★☆☆）	119
8.5. 风险管理（次重点★★★☆☆）	119
9. 计算机组成原理（次重点★★★☆☆）	120
9.1. 计算机系统层次结构（次重点★★★☆☆）	120
9.2. 数据表示与编码（次重点★★★☆☆）	123
9.3. 存储器系统（次重点★★★☆☆）	126
9.4. 输入输出系统（次重点★★★☆☆）	132

9.5. 指令系统（次重点★★★☆☆）	133
9.6. 流水线技术（重点★★★★★）	134
9.7. 阿姆达尔解决方案（次重点★★★☆☆）	136
9.8. 系统性能评估（次重点★★★☆☆）	136
10. 计算机网络（次重点★★★☆☆）	137
10.1. 数据通信基础（次重点★★★☆☆）	137
10.2. 网络体系结构与协议（次重点★★★☆☆）	142
10.3. 常见的应用层协议（次重点★★★☆☆）	144
10.4. 网络地址（次重点★★★☆☆）	146
10.5. 子网和子网掩码（次重点★★★☆☆）	147
10.6. 无分类域间路由（CIDR）	149
10.7. IPv4 VS IPv6（次重点★★★☆☆）	150
10.8. 局域网（次重点★★★☆☆）	151
10.9. 无线局域网（次重点★★★☆☆）	153
10.10. 网络互连与常用设备（重点★★★★★）	154
10.11. 网络工程（次重点★★★☆☆）	155
11. 操作系统（次重点★★★☆☆）	156
11.1. 进程管理（次重点★★★☆☆）	156
11.2. 内存管理（重点★★★★★）	163
11.3. 文件系统（次重点★★★☆☆）	166
12. 数据库系统（超级重点★★★★★）	169
12.1. 数据库模式（重点★★★★★）	169
12.2. 关系模型（重点★★★★★）	171

12.3. 数据库设计与建模（超级重点★★★★★）	177
12.4. 概念设计（超级重点★★★★★）	177
12.5. 逻辑设计（超级重点★★★★★）	180
12.6. 数据库的控制功能（重点★★★★★）	192
12.7. 数据库性能优化（次重点★★★☆☆）	196
12.8. 备份与恢复技术（次重点★★★☆☆）	199
12.9. 数据仓库技术（次重点★★★☆☆）	200
12.10. 分布式数据库（次重点★★★☆☆）	201
12.11. 数据分片（次重点★★★☆☆）	202
13. 信息安全（次重点★★★☆☆）	203
13.1. 信息系统安全体系（次重点★★★☆☆）	203
13.2. 数据加密技术（重点★★★★★）	204
13.3. 认证技术（重点★★★★★）	205
13.4. 密钥管理体系（次重点★★★☆☆）	209
13.5. 通信与网络安全技术（次重点★★★☆☆）	209
13.6. 系统访问控制技术（次重点★★★☆☆）	210
14. 嵌入式系统分析与设计（次重点★★★☆☆）	212
14.1. 嵌入式系统概述（重点★★★★★）	212
14.2. 嵌入式数据库系统（重点★★★★★）	214
14.3. 嵌入式操作系统（重点★★★★★）	216
14.4. 多任务调度算法（次重点★★★☆☆）	217
15. 法律法规（重点★★★★★）	220
15.1. 保护对象（重点★★★★★）	221

15.2. 保护期限（重点★★★★★）	221
15.3. 职务作品（重点★★★★★）	222
15.4. 侵权判定（重点★★★★★）	224
15.5. 专利法（补充重点★★★★★）	224
16. 后记	225

前言

版权提醒：本资料已通过国家版权局登记（登记号：渝作登字-2025-A-00624260），（一旦发现资料恶意流出，直接封号不退款，后果严重的话会请求平台披露你的个人信息，并于互联网法院起诉，请珍惜账号权益和个人名誉，谢谢）。

各位会员同学好，这是凯恩自己结合大量资料（官方教材讲义、历年真题、专业教材）和自己对于考试的理解，研究编制的系统架构设计师和系统分析师公共知识的内部讲义。有的同学知道，凯恩之前的资料架构和系分放在一起的，但是这样有很多不便。比如同学会问某个章节到底要不要看，会有很多迷惑的地方。所以干脆把架构和系分红宝书还是拆开来。

除了形式上的变化，另外在内容上，不单单是书本的内容了，还有专业教材和其他有效的互联网资料。因为软考的教材拼凑感严重，逻辑跳跃比较严重，有时候上下文都没办法联系起来，所以需要搜集大量其他专业教材来让它变得更加科学和系统。和之前的红宝书版本一样，为了方便大家快速突破重难点，凯恩对关键的内容进行了标注，其中下划线标出的内容非常有可能在选择题中考试到，必须熟悉起来，其他内容形成表格帮助大家快速梳理相关的核心概念，用于短时间突破选择题。

再来说点题外话，软考最牛的一点在于以考代评和没有有效期限限制。对于一个以考代评的考试，实际上是要体现传统评审的东西的，其中最主要的就是体现你的业绩。传统评审主要看你参与的项目论文（一般需要单位认可），造假成本比较高（这个不多说懂的都懂）。而以考代评的考试，目前考试的内容和考试形式没法做到对你业绩真实性进行认定。这意味着，即使你手头没有任何资源（没做过项目没做过架构师没做过系统分析师），也可以在论文里说自己从事架构设计师/系统分析师的岗位，编一些内容来糊弄阅卷人，只要让他相信你做过就行。在官方的教材中，其实默许了这个行为（如下图）。

1. 加强学习

根据自身经验的多少，可以采取不同的学习方法。

(1) 经验丰富的报考人员。主要是将自己的经验进行整理，从技术、管理、经济方面等多个角度对自己做过的项目进行归纳、剖析、抽象和升华，在总结的基础上，结合专业知识和关键技术进行分类和梳理。

(2) 经验欠缺的在职开发人员。可以通过阅读、学习、整理单位现有的文档、案例，同时参考历届考题进行学习。思考别人是如何站在系统分析师角度考虑问题的，同时可以采取临摹的方式提高自己的写作能力和思考能力。这类人员学习的重心应放在自己欠缺的方面，力求全面把握。

(3) 学生。学生的特点是有充足的时间用于学习，但缺点是实践经验相对较少，对于这类考生来说，论文考试的难度比较大。从撰写的论文分析，学生在系统分析师的考试中，论文的内容容易空洞而不切实际。因此，作为学生考生，要想更好地完成论文题目，就需要大量地阅读相关文章和论文范文，把别人的直接经验变为自己的间接经验，并进行强化练习。

除非以后考试改革，取消论文写作，否则永远不会对你的论文真实性进行考察（成本太高也没必要）。这就相当于软考让获取职称的方式变简单了（以考代评不花钱不求人），这对普通人来说就是一个超级无敌巨大的利好。我们普通人不知道什么时候软考会改革，不知道以后以考代评会不会退化成考评结合。普通人要做的，能做的就是赶紧上岸赶紧拿证。体制内的同学祈祷早点聘用，体制外的同学早点领取人才补贴落袋为安。

使用说明

(1) 初期筑基。先看这个红宝书一本全。红宝书总计 200 页，合计 11 万字。内容基本覆盖所有综合题（选择题）考试细节，配合我的公共课视频（综合知识视频）特别适合第一轮系统搭建知识框架。通过这份资料可以帮助考生吃透选择题考点。但是在二次复习的时候，有的同学觉得罗嗦，想要一个 lite 版本，纯背的那种。第一出两个版本维护成本很高，第二其实是没必要。等你把这里的资料翻烂了，你会看得一遍比一遍快，你心中就会诞生一个 lite 版本。

(2) 重点导航。凯恩对各个知识点进行了标注，其中标注重点的，属于重要知识点。重点内容可以说不看必挂！标注超级重点的那就是在选择题和案例题都会考的，一处背了两个科目都可以用。次重点章节可看可不看，但是真题还是要做要会。

红宝书不要试图一次逐字看完，一次性记住。这大错特错！凯恩建议先配合我的 B 站视频（搜索芝士架构凯恩）快速扫完一遍。假如遇到非重点章节，真题刷到再回过头来看。这样重复 3-4 回

就可以上考场了。相信我，跟着我这样学，你的选择题大概率没问题。这里要记得是一定要配合刷题，否则无法学以致靠考，效果很差。

【关键提醒】

每学完 1 章必须完成对应真题训练（推荐「芝士架构」APP 每道题我都看过好几次确保解析没问题），否则知识留存率下降 60%。对红宝书标注或真题解析存疑的，建议通过微信私信凯恩提交具体题号+困惑点，可获得定制化解题路径分析。下面是一个基本的备考方案，建议大家根据自己的实际情况进行调整。

阶段名称	周期	核心任务	具体步骤	工具/方法	注意事项
阶段一：速通筑基	15 天	快速搭建知识框架	每日观看凯恩 B 站「芝士架构」视频（1.5 倍速） 精读红宝书★星标重点章节 非重点章节仅浏览小标题	B 站视频 公共知识红宝书	单日学习量≤10 页 非重点章节只需建立印象，无需深读
阶段二：真题淬炼	15 天	真题实战与错题沉淀	完成「做真题→查红宝书→建错题本」闭环 标注真题中冷门考点并补充到红宝书 通读次重点章节	历年真题 红宝书 错题本	冷门考点用荧光笔标注 次重点章节通读后理解度需达 70%
阶段三：循环迭代	3-4 轮（15 天）	多维度强化记忆	每轮侧重不同维度	错题本（APP） 红宝书	首轮：构建知识体系框架 次轮：聚焦真题高频考点 末轮：集中攻克错题集内容
阶段四：冲刺提效	考前 7 天	考点浓缩与实战模拟	制作思维导图（含★重点） 分析解题逻辑与红宝书关联	XMind 工具	模考严格计时（上午/下午各 1 次） 错题需标注对应红宝书页码及考点逻辑

常见问题

Q1：有了红宝书还用看书吗？

实际上红宝书的内容是远大于书本知识体系的，所以红宝书就够了。但是红宝书的内容仍然不能覆盖所有的考点。因为不管是系统架构设计师还是系统分析师，实际考试的内容是一个非常庞大的知识体系，考察的内容非常广泛，我们只能抓重点，抓主干进行学习。假如你想胡子眉毛一把抓，就必须要把书从头看到尾，从头背到尾。这种方式时间成本太高，对于大部分打工人来说不现实，另外实际考题中，书本上的内容实际上就重点考察若干章，如此大费周章，投产比 ROI 太低。相信我，只要红宝书掌握，真题掌握，加上你的积累（后端开发经验/项目参与经验），一次合格的概率极大。

Q2: 红宝书背出就稳了吗？

不一定。前面说了，目前特别是案例部分，变数很大。从真题角度来看，它不喜欢考常规，不喜欢考书本内容。因此这个方面的知识红宝书会尽可能猜测，尽可能补充，但是不能说 100% 覆盖。所以你平时需要有一定的后端技术积累（架构考生）或者一定的信息化项目参与经验（系分），否则无法应付。从 24 年 11 月开始，案例的阅卷故意压分非常明显。这就意味着可能你觉得自己答得还行，和别人一对答案，百度一搜索，觉得会拿一部分分。这是因为实际阅卷过程中要控制通过率，所以会临时提高阅卷标准（这个已经私下和阅卷人核实过了）。所以你要在考场上动用你所有能力，把想到的都写上去，把答案丰富起来，才能取得好成绩。

Q3: 红宝书和其他辅导资料相比，优势在哪里？

红宝书通常是经过凯恩精心整理编写迭代了 N 多个版本（目前已经 6 个大版本），对考试大纲的覆盖度较高，知识点讲解较为系统全面。凯恩通过将复杂的知识体系进行梳理，以更清晰易懂的方式呈现给考生，帮助考生构建完整的知识框架。

Q4: 学习红宝书的时候，有没有什么高效的记忆方法？

可以采用分模块记忆的方式，将红宝书的内容按照知识板块划分，逐个击破。特别是标注重点的章节，一定要结合笔记、思维导图来强化理解和记忆，把重点、难点以及自己的理解感悟记录下来，通过思维导图梳理知识点之间的逻辑关系，背诵记忆口诀提高记忆效率。另外，你可以利用艾

宾浩斯遗忘曲线，合理安排复习时间。

Q5: 除了红宝书和真题，还有哪些学习资料对备考有帮助？

相关专业的经典教材是很好的补充资料，例如关于软件工程、计算机网络等基础学科的权威教材，能帮助你深入理解底层知识。行业内的技术博客、论坛也是不错的信息来源。这个适合有时间有余力的同学学习。并且只是补充，而不是主要的复习资料来源。

Q6: 网上的一些免费学习资料可以用吗？

网上的免费学习资料可以作为辅助，但要注意筛选。有些免费资料，特别是论文范文这种曝光度很高的，你可以学底层逻辑，但是不要去抄。那么红宝书范文也不能抄？不能。也是同样的道理。你只能去用 AI 助手生成。

Q7: 红宝书和 APP 为什么不参照书本大纲进行编排？

真实的考试中不同的内容侧重点和深度都不同，假如按照大纲来你会发现有的知识点无法归类。所以红宝书是根据考试知识点来进行分类整理的，这样可以过滤不少低频的无用的信息。

Q8: 红宝书知识点没有联系，看不懂怎么办？

很遗憾，软考考试内容多，假如放在大学期末考那至少整个 4 门专业课。所以知识点很多，红宝书做不到也不打算把知识点写的很详细。因为知识点太多了，展开写的话 100 万字都打不住。所以也不要指望单看红宝书就能掌握全部的选择，全部的案例，还是要配合 B 站芝士架构凯恩的视频+刷题去理解。

Q9: 我看了红宝书看了视频，选择题还是不会做，为什么？

不会做正常，会做不正常。因为软考选择范围考察极大，全部覆盖我估计要 100 万字内容。这些内容你要再短时间内全部记住显然不现实。所以芝士架构推出的红宝书+视频解决的是关键的知识、高频的内容（没错红宝书 10 万字，案例冲刺 7 万字只能解决高频考点，大概占比 40%-50%），剩下的内容你只能通过自己的积累了。

这里又引出一个问题，为什么选择题那么变态，为什么要那么多超纲的。换位思考，假如你是

出题人，软考办给你的任务是每次考试只能让 5%-10% 的人通过，你会怎么做。会不会给考生全部的考点，重点，会不会就一个题库里这些老题反复考？显然不会。当然你会说，可以通过主观题控分呀。但是主观题控分多麻烦，软考办每次都要和各个阅卷中心沟通协调，还不如提高选择题的难度，先卡掉一部分人再说。

但是话说回来，这种抽象的出题模式对有工作经验的，基础比较好的同学非常友好。你只要掌握核心内容，剩下的就拼基础，不用卷死卷活。看看隔壁高项，案例、论文题目都是有知识点范围限制的，最后就在主观阅卷部分强行卡你，压你分，通过率奇低无比。

在线刷题

想要检测你的学习效果推荐你去使用我们的 PC 刷题网站 www.cheko.cc。看到这个资料的同学可以用上面的错题筛选，每日一练，搜索，真题导出，论文助手等功能。



或者 APP 刷题应用市场搜索芝士架构（蓝白图标的就是），和 Web 版本是同步的。



红宝书 6.0 章节安排思路

软件工程概述这一章节实际上对应了架构设计师教材第二版上第五章软件工程基础知识中部分章节。实际上软件工程不单单是包括开发方法，软件生命周期，还要包括需求分析、系统设计、架构设计、软件测试、部署运维、项目管理等多方面内容。但是，由于软考高级对于这些知识的考察都比较细致，远超书本所提及的内容，假如都放在一起，那么这一章节过于臃肿，层级会很深，

不方便阅读。所以在这里，我把软件需求工程、系统设计、系统架构设计、项目管理、软件测试、系统运行与维护，按照软件生命周期的顺序，拆解成独立的章节，进一步完善和补充考试要求的相关知识，方便你更好的阅读和学习。软件工程这些内容没有计算，纯概念题，基本只在选择或者论文出现，也是属于不看必挂的内容。

1. 软件工程概述（重点★★★★★）

软件工程概述主要在选择题、论文题中都有考察。这一章非常枯燥，都是软件理论，直接听我视频课，过一遍就行。建议通过我的思维导图去回忆，配合真题加深记忆。对于综合题来说，本章所有下划线标出的内容都要熟悉，包括生命周期，开发方法，开发模型，开发环境和工具，软件过程管理等，这些概念需要配合刷题进行记忆。在架构考试中，软工考察得比较少，可以不关注。论文部分只关注一个是敏捷，一个是 RUP，另一个是测试开发，具体以论文押题为准。

1.1. 软件工程相关定义（次重点★★★☆☆）

软件工程定义从不同的角度出发就会有不同的概念，所以这个一点也不重要。凯恩建议你只要记住这里的 PDCA 循环即可（因为选择题考过一次）。

软件工程由方法、工具和过程三个部分组成。包括以下 4 个方面 PDCA。（1）P（Plan）——软件规格说明。规定软件的功能及其运行时的限制。（2）D（Do）——软件开发。开发出满足规格说明的软件。（3）C（Check）——软件确认。确认开发的软件能够满足用户的需求。（4）A（Action）——软件演进。软件在运行过程中不断改进以满足客户新的需求。

1.2. 软件开发方法（重点★★★★★）

实际考试不会来问你如何划分软件开发方法。但是这里的内容，实际可以看作是后面章节内容的目录，故凯恩这里仍然保留。

分类方法	具体方法	描述
开发风格	自顶向下方法	系统的整体结构首先被定义和设计，然后逐步细化为更具体的模块和功能
	自底向上方法	是一种从具体模块开始逐步构建整个系统的方法
性质	形式化方法	形式化方法是一种具有坚实数学基础的方法
	非形式化方法	非形式化方法则不把严格性作为其主要着眼点
适应范围	整体性方法	适用于软件开发全过程的方法称为整体性方法
	局部性方法	适用于开发过程某个具体阶段的软件方法称为局部性方法

1.2.1.形式化方法（非重点☆☆☆☆☆）

形式化方法是指在软件开发过程中，使用严格的数学模型和形式化语言来描述系统的需求、设计和行为，并通过逻辑推理或模型检测等形式化验证手段，证明软件在所有可能的运行情况下都满足既定规范，从而获得可证明的软件正确性。它关注的不仅是测试能否发现错误，而是从理论上保证程序在功能、行为、安全性和活性等方面“必然正确”，常用于安全攸关或高可靠性系统的软件开发与验证。

这里需要掌握形式化方法的特点。（1）正确性验证难且耗时，数学模型和证明困难。（2）仍然需要和传统测试相结合。形式化方法提供理论保证，而测试则负责实际运行和发现运行时错误，不能说有了形式化方法，软件开发就不需要测试了。

1.2.2.净室软件工程（次重点★★★★☆）

净室软件工程（Cleanroom Software Engineering，CSE）首先是一种上面提到的“形式化方法”，这一点非常关键。它不是测试方法、不是管理模型、也不是单纯的过程模型，而是强调用数学与统计理论，在设计和规约阶段消除错误的软件工程方法。它的理论基础是函数理论和抽样理论，如下表所示。

理论基础	展开描述
函数理论	函数理论净室软件工程的首要理论基础是函数理论，其核心主张将程序本质定义为从输入集合（定义域）到输出集合（值域）的严格映射关系，因此程序规约与函数规约具有完全等价性。围绕这一理论，相关考核常聚焦三大核心性质：一是完备性，二是一致性，三是正确性（辅助记忆：玩一阵）。
抽样理论	净室软件工程的第二大理论基础是抽样理论，其核心逻辑源于软件测试的固有局限：由于软件输入空间的无限性，穷尽所有可能场景的测试在实践中无法实现。基于此，净室方法引入统计学思想，将所有潜在使用场景视为整体样本空间，通过科学抽样选取代表性测试用例，进而推断整个系统的可靠性水平。

既然是软件开发过程，那么他也有开发步骤和测试步骤，主要关注其中四个，了解即可。统计过程控制下的增量式开发、基于函数的规约 + 盒子结构方法、正确性验证、统计测试与软件认证。

这里需要掌握的是，净室并非软件研发的“银弹”，其应用受技术门槛、实践场景等多重因素限制。（1）其正确性验证依赖严格的理论推导与数学基础，对工程师专业能力要求极高且验证成本显著。（2）净室不依赖传统单元测试（但还是需要传统的模块测试），但其工程实践中仍需考虑编译器、操作系统等底层环境缺陷。（3）净室本质仍属于传统软件工程体系，并未突破该体系的固有边界，无法解决所有工程问题。

1.2.3.逆向工程（次重点★★★★☆☆）

次重点章节，出过 3-4 次选择。这里要区分逆向工程的几个概念。特别是再工程和逆向工程的区别一定要掌握。选择题考了多次。

概念	说人话	定义
逆向工程	从低层（代码）推回高层设计	分析程序，在比源代码更高抽象层次上建立程序表示，是设计恢复过程（注意和再工程的区分）
重构	同一抽象层里改结构，不升降层次	在同一抽象级别上转换系统描述形式
设计恢复	借助工具抽取设计信息	借助工具从已有程序中抽象出数据设计、总体结构设计和过程设计等信息

再工程（必背）	逆向工程 + 新需求 + 正向工程	在逆向工程信息基础上，修改或重构已有系统产生新版本，包括逆向工程、新需求考虑和正向工程三步
正向工程	从设计到代码	从现有系统恢复设计信息，用此信息改变或重构系统，改善整体质量（这个书上是放在逆向工程的语境下来说的，实际上我们都知道正向工程都是从零开始，这里主要起强调的是利用逆向工程的信息，解答逆向工程的作用）

这里要注意逆向工程和再工程的区别。逆向工程是从代码、程序这些低层内容出发，反推出系统的设计、结构、流程和文档，重点是分析和恢复设计，不一定修改系统。而再工程是看懂旧系统以后，再改造升级。所以考试看到“分析程序、恢复设计”，选逆向工程；看到“修改、重构、升级、产生新版本”，选再工程。

不同的逆向工程抽象层次从低到高如下表所示，要记忆。

实现级	包括程序的抽象语法树、符号表等信息，最低级的抽象
结构级	包括反映程序分量相互依赖关系信息，如调用图、结构图等
功能级	包括反映程序段功能及程序段之间关系的信息
领域级	包括反映程序分量或实体与应用领域概念对应关系的信息，最高级的抽象

这里每个层次反映出来的到底是哪些信息必须要利用下面的口诀加以记忆，选择题必考。

记忆口诀：世仇结分工龄（世仇结婚还要计算工龄）。世（实现）仇（抽象信息）结（结构）分（程序分量）工（功能级）龄（领域级）。功能级和领域级包含的信息就是功能和领域所以不单独展开。

1.2.4.基于构件的软件工程 CBSE（超级重点★★★★★）

超级重点章节，这一章不看不背就和那些裸考的一样，没区别。重要之处在于选择、论文反复考，不会不行。构件这块 23 年架构出了至少 5 分选择题。需要重点关注。

进入 1990 年代，随着分布式对象技术的发展，业界对独立部署、组装软件单元的需求愈发强烈。另一位 UML 创建者 Ivar Jacobson 在 1993 年将构件描述为“已经实现、用于增强编程语言

构造的单元”也体现了早期对构件的理解。直到组件化软件工程（CBSE）兴起，软件构件的概念才得到系统深入的定义。基于构件的软件工程（Component-Based Software Engineering, CBSE）是一种新型的软件开发模式，旨在通过复用可重用的软件构件来构造高质量、高效率的应用软件系统。这种模式强调将软件系统分解为独立的构件，每个构件都具有明确定义的功能和接口，可以独立开发、测试和部署。

1.2.4.1. 构件的定义（重点★★★★★）

书上没有提到，实际上，这是伊恩·萨默维尔提出的构件的定义，选择喜欢考，所以需要尊重一下。这里唯一要补充的是可见状态，参见下表的解释。

构件特性	描述
可组装性	对于可组装的构件，所有外部交互必须通过公开定义的接口进行。另外，它还必须提供对自身信息的外部访问，例如它的方法和属性
可部署性	为使之可部署，构件需要是自包含的，它必须能作为一个独立实体在提供其构件模型实现的构件平台上运行。因而意味着构件总是二进制形式的且无须在部署前编译。如果一个构件实现为一项服务，它不必由用户来部署，而是由服务的提供者来部署
文档化	构件必须是完全文档化的，这样所有用户能确定是否构件满足他们的需要。应该定义所有构件接口的语法甚至语义
独立性	构件应该是独立的，它应该可以在无其他特殊构件的情况下进行组装和部署。如果在某些情况下构件需要外部提供的服务，应该在“请求”接口描述中显式地声明
标准化	构件标准化意味着在 CBSE 过程中使用的构件必须符合某种标准化的构件模型。此模型会定义构件接口、构件元数据、文档管理、组成以及部署
可见性	没有（外部的）可见状态（这里包含两个特殊情况，1.构件本身不直接暴露外部可见状态，但可以通过运行时容器（如应用服务器、框架等）管理状态并对外提供可见性，2.对于不影响构件功能的某些属性可以对外部可见）

记忆口诀：竹鼠挡毒标。竹（可组装性）鼠（可部署性）档（文档化）毒（独立性）镖（标准化）。

1.2.4.2. 构件分类（次重点★★★★☆）

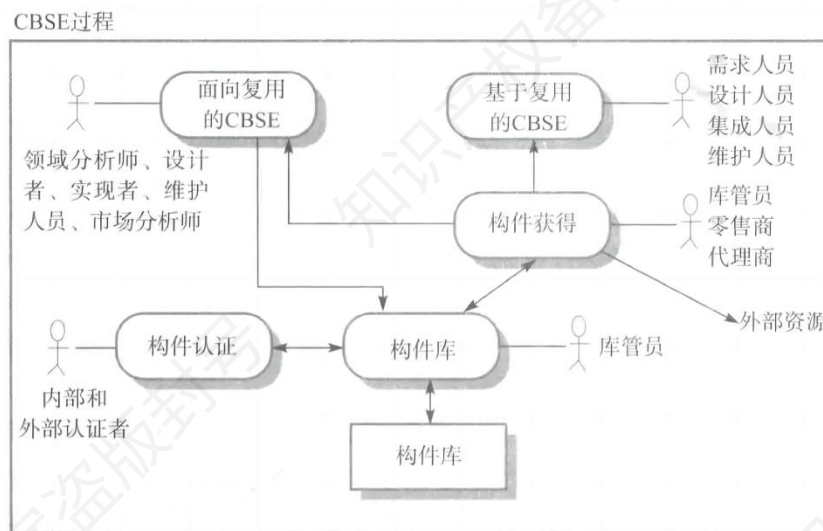
从构件的外部形态来看，构件可分为五类，有一年选择题考察过。

名称	内容	例子
独立而成熟的构件	成熟度高，已在实际环境多次检验；完全隐藏所有接口，用户只需通过规定命令使用。	数据库管理系统、操作系统
有限制的构件	提供接口并指出使用条件和前提；装配时可能产生资源冲突、覆盖等问题，需测试后使用。	面向对象程序设计语言的基础类库
适应性构件	通过包装或接口技术处理了不兼容性、资源冲突等问题，可直接在各种环境中使用。	ActiveX 组件
装配的构件	安装时已装配在操作系统、数据库管理系统或信息系统不同层次上，使用胶水代码即可连接。	多数软件商提供的成品软件产品
可修改的构件	可修改错误、增加新功能，通过重新“包装”或写接口实现构件版本替换。	应用系统开发中常用的功能模块

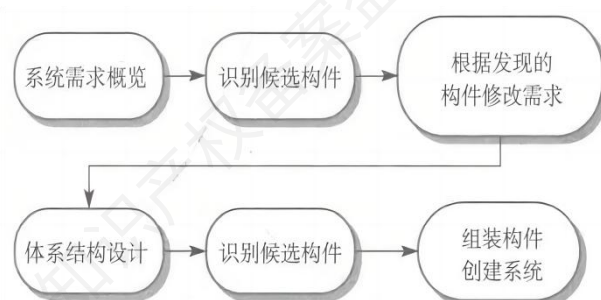
记忆口诀：独（独立）显（限制）时（适应）装（装配）秀（修改）。独立显卡挂在衣服上走时装秀。

1.2.4.3.CBSE 过程的主要活动（次重点★★★☆☆）

实际上按照伊恩·萨默维尔的理论，CBSE 活动分为两块。一个面向复用的开发。这个过程是开发将被复用在其他应用程序中的构件或服务。它通常是对已存在的构件进行通用化处理。另一个是基于复用的开发。这个过程是复用已存在的构件和服务来开发新的应用程序。书本上其实说的是后者。下面是两块活动的图例。



基于复用的开发过程如下所示，最初用户需求概要灵活，需完整以识别复用构件；早期使用可用构件细化修改需求，若不满足让用户改需求要么自己改（让他权衡成本这个是比较违反直觉的）；系统体系结构设计后需进一步构件搜索与设计精化，可能要找替代方案并修改构件功能；构件集成组装来开发系统，包括与基础设施集成、开发适配器等。



有的同学指出上图有误，指出下方的“识别候选构件”有误，应该是定制和适配，如下所示。

- (1) 系统需求概览；(2) 识别候选构件；(3) 根据发现的构件修改需求；(4) 体系结构设计；(5) 构件定制与适配；(6) 组装构件，创建系统。

实际上上图出自伊恩·萨默维尔的软件工程著作中的图例，在那本书的上下文里把下面的“识别候选构件”解释为构件定制与适配，但是写还是写成“识别候选构件”。

1.2.4.4. 构件组装（次重点★★★☆☆）

组装方式选择题常考，所以还是要背，关键是这三种组装的描述，给你描述要知道属于什么，

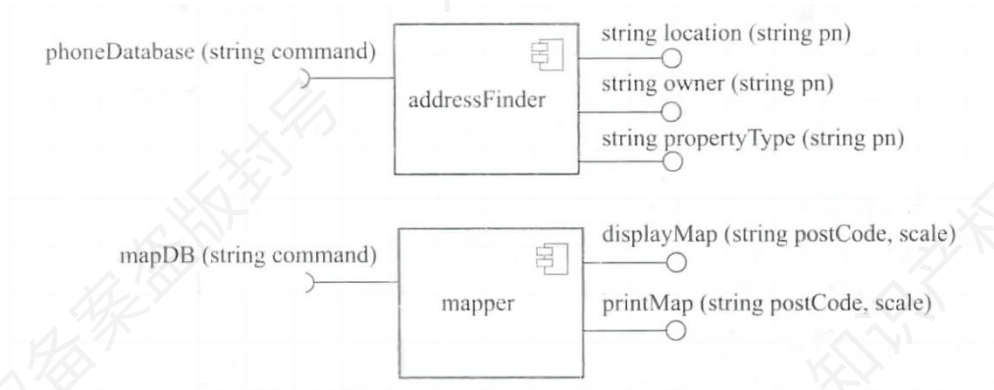
给你名称要知道具体的描述是什么。

组装类型	描述	特点	图示
顺序组装	通过按顺序调用已有构件创造新构件，看作“提供”接口的组装，先调用构件 A 服务，用 A 结果调用构件 B 服务	构件间不相互调用，可能被外部程序调用，需特定胶水代码确保接口兼容	
层次组装	一个构件直接调用另一个构件提供的服务，被调用构件“提供”接口与调用构件“请求”接口需兼容	接口匹配无需额外代码，不匹配需转换代码，构件作为网络服务时不能用	
叠加组装	两个或以上构件叠加创建新构件，新构件接口是原构件接口组合，通过外部接口分别调用原构件	A 和 B 不相互依赖和调用	

在组装构件时可能会遇到接口不兼容的问题，三种情况的判别要记忆。这里凯恩也给出了一个例子，方便你理解什么叫做不兼容。

不兼容类型	描述
参数不兼容	接口两侧操作名相同，但参数类型或数目不同

操作不兼容	“提供” 接口和 “请求” 接口操作名不同
操作不完备	一个构件的 “提供” 接口是另一个构件的 “请求” 接口的子集，或相反



上面是一个构件图，其中

提供接口：表示构件向外部提供的服务。在图形上，它用一个圆形来表示，连接在构件的边缘。从构件引出的线条连接到这个圆形，表示构件通过这个接口提供服务。这里的 **location**，**owner** 方法都是服务，括号里是他们的传入参数。

请求接口：代表构件在实现其功能时需要调用外部的服务。在图形上，它用一个半箭头来表示，同样连接在构件的边缘。箭头指向外部，表示构件需要从外部获取相应的服务。例如这里 **phoneDatabase** 就是 **addressFinder** 需要调用的外部服务。

这里说个题外话，实际上考虑构件的兼容与否的前提是要对构件做层次组装，换句话说，只有接口兼容才能做层次组装，或者说我们要做层次组装的前提是接口兼容。假如你要做的是顺序或者叠加实际上有中间代码参与可以做兼容和转换。以上图为例，因为 **addressFinder** 提供的接口和 **mapper** 请求的接口不一样，即 **addressFinder** 的输出不能作为 **mapper** 的输入，属于操作不兼容，既然操作都不兼容参数兼容也谈不上，也就不能做层次调用，需要通过叠加组装或者顺序组装添加胶水代码来做转换。

1.3.软件过程/生命周期模型（重点★★★★★）

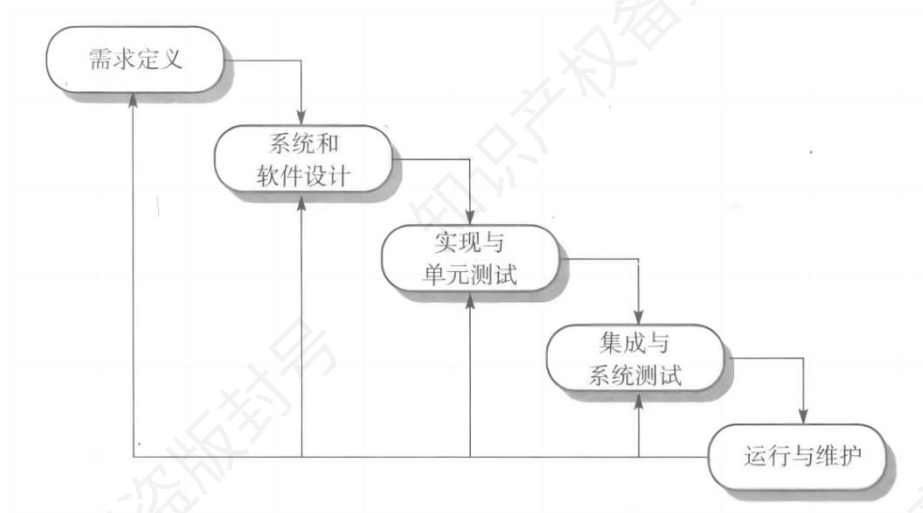
软件开发模型这一章是超级重点，选择爱考，有一年系分案例也着重考察了。经过凯恩的调研，得出的结论是软件过程 = 软件开发生命周期模型。它们是同义词。这点要知道，以免在不同的上下文给你理解带来歧义。

软件开发模型给出了软件开发活动各阶段之间的关系，它是软件开发过程的概括，是软件工程的重要内容。软件要经历从软件定义、软件开发、软件运行、软件维护（软件生命周期方法学），直至被淘汰这样的全过程，这个全过程称为软件的生命周期。软件生命周期描述了软件从生到死的全过程。为了使软件生命周期中的各项任务能够有序地按照规程进行，需要一定的工作模型对各项任务给予规程约束，这样的工作模型被称为软件过程（开发）模型，有时也称之为软件生命周期模型。下面的表格，凯恩按照不同的场景对场景的开发模型进行了分类。

前提条件	具体模型示例
软件需求完全确定	瀑布模型
软件开发初始阶段仅能提供基本需求	喷泉模型、螺旋模型、统一开发过程、敏捷方法等
以形式化开发方法为基础	变换模型

1.3.1.瀑布模型（重点★★★★★）

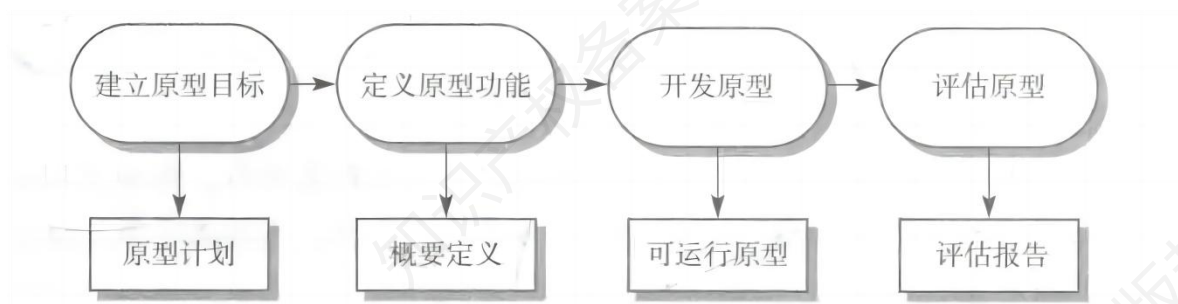
瀑布模型最早由 Winston W. Royce 在 1970 年的一篇论文中提出，它将软件开发的过程分为需求分析和定义、系统和软件设计、实现和单元测试、集成和系统测试和运行维护 6 个阶段，形如瀑布流水，最终得到软件产品，如下图所示。我视频里是另外一个版本的图片和下图不一样，不影响理解，不会考你原图，关键是要知道瀑布模型的缺点。



缺点：依赖于早期进行的需求调查，不能适应需求的变化；软件需求的完整性、正确性等很难确定，甚至是不可能和不现实的。

1.3.2.原型模型/演化模型/快速原型模型（重点★★★★★）

由于瀑布型的缺点，人们提出了原型模型。该模型如下图所示。



原型模型/演化模型/快速原型模型软考里一般不做区分，当做一个东西就行。它们都是适用于事先需求难完整定义的软件开发，基于快速开发的原型，依据用户反馈改进，不断迭代直至形成最终产品。

其优点是功能开发后可及时测试以验证需求，助于产生高质量要求；缺点是若让用户过早接触不稳定功能，可能对开发人员和用户造成负面影响。原型的特点是迭代，和下面增量模型有区分。

1.3.3.螺旋模型（重点★★★★★）

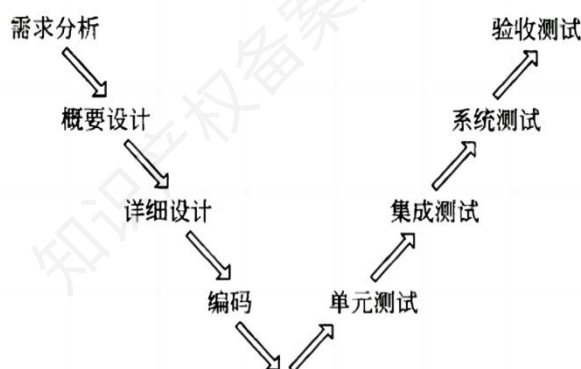
螺旋模型这里关键是要记忆，螺旋模型 = 快速原型 + 风险分析，选择题特别爱考。教材中

1.3.5.变换模型（次重点★★★★☆）

变换模型是基于形式化规格说明语言和程序变换的软件开发模型，需要严格的数学理论和一整套开发环境的支持。

1.3.6.V 模型（重点★★★★★）

V 字模型常考什么设计是什么测试的依据，比如问你概要设计的产物是集成测试还是系统测试的依据，直接看下图你会认为概要设计和系统测试是对应的。好问题来了。系统测试的设计依据是什么？是概要设计？非也，实际上概要设计的产物实际上是集成测试的依据。集成测试人员可以根据概要接口定义来设计测试用例。有的同学会问，照你这么说，那么验收测试的依据是什么？那就是用户需求。



这里顺带提一下软考常考的三种测试类型，特别是测试依据需要清楚。

测试类型	测试对象	测试目的	依据（文档）
单元测试	可独立编译或汇编的程序模块、软件构件或 OO 软件中的类（统称为模块）	检查每个模块能否正确实现设计说明中的功能、性能、接口和其他设计约束等条件，发现模块内差错	软件详细（设计说明书）
集成测试	模块之间，以及模块和已集成的软件之间的接口关系	检查接口关系，并验证已集成的软件是否符合设计要求	软件概要设计（文档）

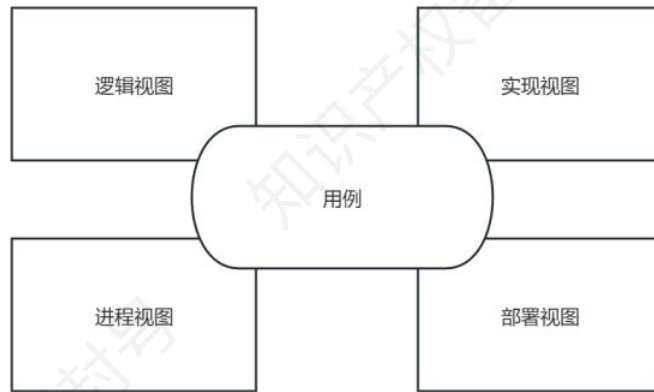
系统测试	完整的、集成的计算机系统	在真实系统工作环境下，验证完整的软件配置项能否和系统正确连接，并满足系统 / 子系统设计文档和软件开发合同规定的要求	需求分析（文档或开发合同）
验收测试	已完成全部开发工作、准备交付用户使用的软件系统	验收测试的目的就是从用户角度验证系统是否满足合同或需求规定的业务目标，确保系统可交付、可上线，并得到用户的正式认可	用户需求

1.3.7.统一过程（超级重点★★★★★）

RUP 在 2000 年代初很流行，但是 RUP 因为重流程、节奏慢、成本高，不适应快速变化的互联网环境，而敏捷以短迭代、轻文档、快速交付和灵活应变成为主流。你可以把 RUP 当成之前的陆地霸主恐龙，现在基本灭绝了而已。但是 RUP 不流行不代表没有意义（它的一些迭代增量的思想实际上对敏捷有很大的影响），不代表软考不考。实际上，软考常考 RUP 特点，生命周期。所以这块内容还是要背。

1.3.7.1.RUP 概述（重点★★★★★）

RUP（Rational Unified Process）将项目管理、业务建模、分析与设计等统一起来，贯穿整个开发过程。RUP 是用例驱动的、以体系结构为中心的、迭代和增量的软件开发过程。RUP 中的开发活动是围绕体系结构展开的。软件体系结构的设计和代码设计无关，也不依赖于具体的程序设计语言。RUP 采用“4+1”视图模型来描述软件系统的体系结构。



这里的和后面的架构 4+1 视图不同注意区别。到这里我们只需要知道，软考里总共有 3 种 4+1 视图，后面会做统一的梳理。

1.3.7.2.生命周期（重点★★★★★）

RUP 软件开发生命周期是一个二维的软件开发模型，RUP 中有 9 个核心工作流，这 9 个核心工作流如下。23 年系分考试直接问你这九大核心工作流是什么，所以必须背。

RUP 涵盖 9 大核心关键词，包括业务建模、需求、分析与设计、实现、测试、部署、配置与变更管理、项目管理、环境。

记忆口诀为：建需分食测。建仓前，需先分好粮食、测好食物够不够。建（建模）需（需求）分（分析）食（实现）测（测试）。

部配变项环。部门配了新工具，遇情况得变步骤，项目每个环节都得盯牢。部（部署）配变（配置变更）项（项目管理）环（环境）。

1.3.8.敏捷方法（超级重点★★★★★）

敏捷常用常考，选择，案例，论文都会考察。选择考概念，案例可能考一个背景，不熟悉敏捷流程不好做，论文也会让你写敏捷论文，所以是超级重点。

敏捷方法相对于“非敏捷”而言，它们更强调开发团队与用户之间的紧密协作、面对面的沟通、频繁交付新的软件版本、紧凑而自我组织型的团队等，也更注重人的作用。敏捷方法是适应型，而

非可预测型。敏捷方法是以人为本，而非以过程为本。敏捷开发是迭代增量式的开发过程。

1.3.8.1.敏捷宣言（重点★★★★★）

敏捷宣言认为，个体和交互胜过过程和工具；可工作的软件胜过大量的文档；客户合作胜过合同谈判；响应变化胜过遵循计划。

1.3.8.2.适用场景（重点★★★★★）

（1）适合于规模较小的项目。（2）敏捷方法适用于需求初步萌芽、尚未明确定型并且快速改变的情况，如果系统有比较高的关键性、可靠性、安全性方面的要求，则可能不完全适合。

记忆口诀：小变。小（规模小）变（快速改变）。

1.3.8.3.主要敏捷方法（重点★★★★★）

常见的敏捷方法有个概念就好，记住方法名很重要。

方法	简介	核心价值观 / 原则
极限编程（XP）	轻量级、灵巧且严谨周密的软件开发方法，强调通过短开发周期和频繁发布提高软件质量和开发团队生活质量	交流、朴素、反馈、勇气；强调团队合作、客户满意度和应对变化，遵循 YAGNI 和 DRY 原则等
水晶系列方法	提倡“机动性的”方法，包含具有共性的核心元素	以人为本
Scrum	侧重于项目管理的迭代式增量软件开发过程	注重团队协作、响应变化、关注商业价值等
特征驱动开发方法（FDD）	迭代的开发模型，强调人、过程和技术三要素	未明确提及特别核心的价值观表述